Agentic RAG Internship Project Documentation

1. Chosen Use-case

This project implements an **Invoice-to-Purchase Order (PO) Matcher** system designed to explain why specific invoices are flagged during processing. The system retrieves the relevant purchase orders, goods receipt records, and invoice details from local data sources. It generates clear, human-readable explanations including match scores and confidence levels, supports follow-up queries such as "Approve it" by maintaining a short-term memory of context, and maintains detailed audit logs for transparency and traceability.

2. Implementation Details

The project delivers a fully integrated pipeline with the following components:

- **Planner (Agent):** Interprets user queries and orchestrates the workflow.
- **Retriever:** Fetches invoices, corresponding purchase orders, and receipts from local storage.
- **Verifier:** Calculates match scores and confidence levels to verify invoice validity relative to purchase orders.
- **LLM Synthesizer:** Employs a language model to generate coherent, human-friendly explanations based on retrieved evidence and verification results.
- **Memory Module:** Maintains short-term context to support follow-up queries referring to prior invoices.
- **Audit Logger:** Records all decisions and agent outputs during query processing.
- **Web Interface:** A Flask-based UI enables users to input queries and view responses along with conversation history.

3. Known Limitations

- The multi-agent pipeline anticipated in the full project brief has been simplified to a planner-agent single workflow.
- The verification mechanism relies on heuristic scoring rather than advanced machine learning models.
- Data retrieval is restricted to local files; no integration with cloud or external web data sources is implemented.
- The system lacks user authentication and is not configured for production deployment environments.

4. Recommended Next Steps

To enhance and extend the system, consider the following improvements:

- Develop a multi-agent orchestration framework allowing coordination among multiple specialized agents.
- Integrate cloud-based data storage and retrieval services as well as web search engines for broader and richer evidence acquisition.

- Augment the verifier using machine learning techniques for more accurate and reliable match scoring.
- Implement user authentication, role-based access control, and secure communication protocols.
- Deploy the application via scalable infrastructure solutions, such as containerization with Docker and orchestration using Kubernetes, or utilize cloud platforms like AWS Elastic Beanstalk.

5. Development Process and Insights

The project was approached incrementally, focusing on building a stable core system before progressively adding features. Emphasis was placed on reliability, explainability, and modular design to ensure maintainability. Challenges encountered included managing conversation state for follow-up queries requiring short-term memory and integrating verification logic seamlessly with language model synthesis. Throughout the development, the project provided hands-on experience in combining retrieval, verification, synthesis, and user interface design within a cohesive pipeline.